

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Deemed to be University, Bhubaneswar - 751024, Odisha

MID SEMESTER EXAMINATION — SOLUTIONS & MARKING SCHEME

Subject: Data Structures

Time: 1.5 Hours

Semester / Programme: 3rd Semester

Full Marks: 20

Instructions: Answer Question No. 1 (Compulsory) and any THREE from Question No. 2 to 5. Figures in brackets indicate marks.

SECTION - A (Compulsory)

1(a) Calculate the total number of different Binary Search Trees (BST) that can be constructed using $n = 5$ distinct keys. [1]

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Solution & Key Concept:

- Identify the formula for the n -th Catalan number to count distinct BSTs: $C_n = (1 / (n+1)) C_{2n}\{n\}$.
- Substitute $n = 5$ into the formula: $C_5 = (1 / 5+1) C_{10}\{5\} = (1 / 6) \times (10 \times 9 \times 8 \times 7 \times 6 / 5 \times 4 \times 3 \times 2 \times 1)$.
- Calculate $C_{10}\{5\} = 252$.
- Compute $C_5 = (252 / 6) = 42$.

Answer: 42

Marking Rubric: [1 Mark] for correct formula application and final answer 42

1(b) Convert the following infix expression into its equivalent postfix notation: $A + B \times C - (D / E F)$. [1]

Solution & Key Concept:

- Parenthesize according to operator precedence: $((A + (B \times C)) - (D / (E F)))$.
- Move operators to the right of their respective operands.
- Sub-expression 1: $B \times C \rightarrow BC^*$.
- Sub-expression 2: $A + BC^* \rightarrow ABC^*+$.
- Sub-expression 3: $E F \rightarrow EF$.
- Sub-expression 4: $D / EF \rightarrow DEF/$.
- Combine with outer minus: $ABC^*+ - DEF/ \rightarrow ABC^*+DEF/-$.

Answer: $ABC^*+DEF/-$

Marking Rubric: [1 Mark] for correct postfix expression

1(c) State the maximum number of nodes at level l (assuming root is at level 0) in a Binary Tree. [1]

$$N_l = 2^l$$

Solution & Key Concept:

- At level 0 (root), maximum nodes = $2^0 = 1$.
- At level 1, each node can have at most 2 children, giving $2 \times 1 = 2 = 2^1$ nodes.
- Inductively, at any level l , the maximum number of nodes is 2^l .

Answer: 2^l

Marking Rubric: [1 Mark] for correct formula

1(d) Evaluate the address of element $A[15][10]$ in a 2D integer array $A[1020][515]$ stored in Column-Major order, given base address $\{Base\}(A) = 1000$ and each integer occupies 4 bytes. **[1]**

$$\text{Address}(A[i][j]) = \text{Base} + W \times ((i - LB_r) + (j - LB_c) \times M)$$

Solution & Key Concept:

- Identify given parameters: Row lower bound $\{LB\}_r = 10$, Column lower bound $\{LB\}_c = 5$.
- Calculate total rows $M = \{UB\}_r - \{LB\}_r + 1 = 20 - 10 + 1 = 11$.
- Given indices $i = 15, j = 10$, element size $\{W\} = 4$ bytes, and $\{Base\} = 1000$.
- Apply Column-Major formula: $\{Address\} = 1000 + 4 \times ((15 - 10) + (10 - 5) \times 11)$.
- Evaluate brackets: $(5 + 5 \times 11) = 5 + 55 = 60$.
- Final Address = $1000 + 4 \times 60 = 1000 + 240 = 1240$.

Answer: 1240

Marking Rubric: [1 Mark] for correct step-by-step substitution and final address calculation

1(e) What is the primary advantage of a Circular Queue over a Linear Queue implemented using arrays? **[1]**

Solution & Key Concept:

- In a linear queue, once the rear pointer reaches the maximum capacity, no more elements can be inserted even if space is available at the front due to dequeues (false overflow).
- A circular queue connects the last location back to the first location, efficiently reusing vacated spaces and overcoming memory wastage.

Answer: It efficiently reuses space vacated by dequeue operations, preventing false overflow conditions.

Marking Rubric: [1 Mark] for mentioning space reuse / prevention of false overflow.

SECTION - B (Answer any THREE)

Answer any THREE questions from Q.2 to Q.5. Each question carries 5 marks.

Q.2. (a) Write an efficient algorithm/C function to represent a sparse matrix using a 3-tuple (triple) linked list or array representation, and explain its storage advantage. **[2.5]**

Solution for (a):

- A sparse matrix with many zero elements can be compressed by storing only non-zero triplets: $\langle \text{row}, \text{column}, \text{value} \rangle$.
- The first row of the representation typically stores $\langle \text{total_rows}, \text{total_cols}, \text{total_non_zeroes} \rangle$.
- Storage advantage: Instead of allocating $O(m \times n)$ memory for a matrix of size $m \times n$, it requires only $O(k)$ space, where k is the number of non-zero elements.

Result / Conclusion: 3-tuple representation stores only non-zero elements as (row, col, val), reducing space complexity from $O(m \times n)$ to $O(k)$.

Marking Rubric: [1.5 Marks] for structural explanation and tuple format, [1.0 Mark] for storage efficiency analysis.

(b) Convert the infix expression $Q = ((A + B) \times C - (D - E)) \div (F + G)$ into prefix notation using stack-based evaluation or operator precedence method. **[2.5]**

Solution for (b):

- Given Infix: $((A + B) \times C - (D - E)) \div (F + G)$
- Step 1: Convert innermost sub-expressions: $(A+B) \rightarrow +AB$, $(D-E) \rightarrow -DE$, $(F+G) \rightarrow +FG$.
- Step 2: Substitute back into expression: $((+AB \times C) - (-DE)) \div (+FG)$.
- Step 3: Handle multiplication and outer subtraction in the left sub-expression: $(\times (+AB)C) \rightarrow *+ABC$. Then $(-*+ABC) (-DE) \rightarrow -*+ABC-DE$.
- Step 4: Handle division with the right sub-expression: $(\div (*+ABC-DE) (+FG)) \rightarrow /-*+ABC-DE+FG$.

Result / Conclusion: $/*+ABC-DE+FG$

Marking Rubric: [1.0 Mark] for intermediate sub-expression conversions, [1.5 Marks] for correct final prefix expression.

Q.3. (a) Write a complete C function to reverse a singly linked list in-place. Show the pointer manipulations clearly. [3]

Solution for (a):

- Initialize three pointers: `prev = NULL`, `current = head`, `next = NULL`.
- Traverse the linked list while `current != NULL`.
- Inside loop: store `next = current->next`, reverse pointer `current->next = prev`, shift `prev = current`, and `current = next`.
- After loop finishes, update `head = prev`.

Result / Conclusion: `void reverseList(struct Node** head_ref) { struct Node* prev = NULL; struct Node* current = *head_ref; struct Node* next = NULL; while (current != NULL) { next = current->next; current->next = prev; prev = current; current = next; } *head_ref = prev; }`

Marking Rubric: [1.5 Marks] for correct pointer logic inside traversal, [1.5 Marks] for handle initialization and updating head.

(b) Explain how two polynomials in a single variable can be represented using linked lists. Write the structural node definition in C. [2]

Solution for (b):

- A polynomial consists of terms, where each term has a coefficient and an exponent.
- A linked list node can hold fields for `coefficient`, `exponent`, and a `next` pointer to the subsequent term.
- Polynomial addition is performed by traversing both linked lists simultaneously, comparing exponents.

Result / Conclusion: `struct PolyNode { float coeff; int exp; struct PolyNode* next; };`

Marking Rubric: [1.0 Mark] for conceptual explanation, [1.0 Mark] for correct C struct definition.

Q.4. (a) Construct a Binary Tree given its Preorder traversal: `G, B, A, C, F, E, D, H, I` and Inorder traversal: `A, B, C, D, E, F, G, H, I`. [3]

Solution for (a):

- Preorder root is the first element: `G`. Thus, `G` is the root.
- In Inorder traversal, find `G`. Elements to its left (`A, B, C, D, E, F`) form the left subtree, and elements to its right (`H, I`) form the right subtree.
- Left subtree preorder: `B, A, C, F, E, D`, Inorder: `A, B, C, D, E, F`. Root is `B`.
- Continuing recursively: `B` has left child `A`, right subtree with root `C`. `C` has right child `F`. `F` has left subtree `E` (with left child `D`).
- Right subtree of `G` has preorder `H, I` and inorder `H, I`. `H` is root, `I` is right child of `H`.

Result / Conclusion: Tree structure: Root `G`, Left child `B` (with left `A`, right `C` -> right `F` -> left `E` -> left `D`), Right child `H` (with right `I`).

Marking Rubric: [1.5 Marks] for correct step-by-step root identification, [1.5 Marks] for correct final tree topology.

(b) Write a recursive C function to compute the height (or maximum depth) of a given Binary Tree. $\text{Height}(T) = 1 + \max(\text{Height}(T_{\text{left}}), \text{Height}(T_{\text{right}}))$ [2]

Solution for (b):

- Base case: If the node is `NULL`, return `-1` (or `0` depending on convention, here `-1` for edge count).
- Recursively find the height of left subtree and right subtree.
- Return $1 + (\text{left}_{\text{height}}, \text{right}_{\text{height}})$.

Result / Conclusion: `int treeHeight(struct Node* node) { if (node == NULL) return 0; else { int leftHeight = treeHeight(node->left); int rightHeight = treeHeight(node->right); return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1; } }`

Marking Rubric: [1.0 Mark] for base case, [1.0 Mark] for recursive step and max calculation.

- Q.5. (a)** Starting with an empty Binary Search Tree (BST), insert the following keys in the given order: `50, 30, 70, 20, 40, 60, 80`. Draw the resulting BST and perform its Post-order traversal. **[3]**

Solution for (a):

- Insert 50: Root.
- Insert 30: Left of 50.
- Insert 70: Right of 50.
- Insert 20: Left of 30.
- Insert 40: Right of 30.
- Insert 60: Left of 70.
- Insert 80: Right of 70.
- Post-order traversal rule: Left -> Right -> Root.
- Left subtree post-order: 20, 40, 30.
- Right subtree post-order: 60, 80, 70.
- Combining with root: `20, 40, 30, 60, 80, 70, 50`.

Result / Conclusion: Post-order Traversal: `20, 40, 30, 60, 80, 70, 50`

Marking Rubric: [1.5 Marks] for correct BST construction, [1.5 Marks] for correct Post-order traversal sequence.

- (b)** Write a non-recursive (iterative) algorithm to traverse a Binary Tree in In-order using an explicit stack. **[2]**

Solution for (b):

- Initialize a stack and set `curr = root`.
- Run a while loop `while (curr != NULL || !isEmpty(stack))`.
- Inside loop, traverse left as far as possible: push each `curr` to stack and `curr = curr->left`.
- When `curr == NULL`, pop from stack, visit node (`print node->data`), and set `curr = popped\_node->right`.

Result / Conclusion: ```c void inorderIterative(struct Node* root) { struct Stack* s = createStack(); struct Node* curr = root; while (curr != NULL || !isEmpty(s)) { while (curr != NULL) { push(s, curr); curr = curr->left; } curr = pop(s); printf("%d ", curr->data); curr = curr->right; } } ```

Marking Rubric: [1.0 Mark] for stack push logic along left spine, [1.0 Mark] for pop, visit, and moving to right subtree.

--- END OF PAPER ---